근사 알고리즘

최적 답을 구하기 어려울 때, 빨리 답을 구하면서 얼마나 나쁠 수 있는지도 봅니다.

먼저 작은 예로 살펴봅니다

근사 알고리즘

최적 답을 구하기 어려울 때, 빨리 답을 구하면서 얼마나 나쁠 수 있는지도 봅니다.

예: 최적 비용이 10

두 배 이내를 보장한다면 구한 비용은 20 이하여야 합니다.

10-1 허용 배수 이내의 비용인지 확인

구할 것

구한 비용이 허용한 배수 안에 들어오는지 확인합니다.

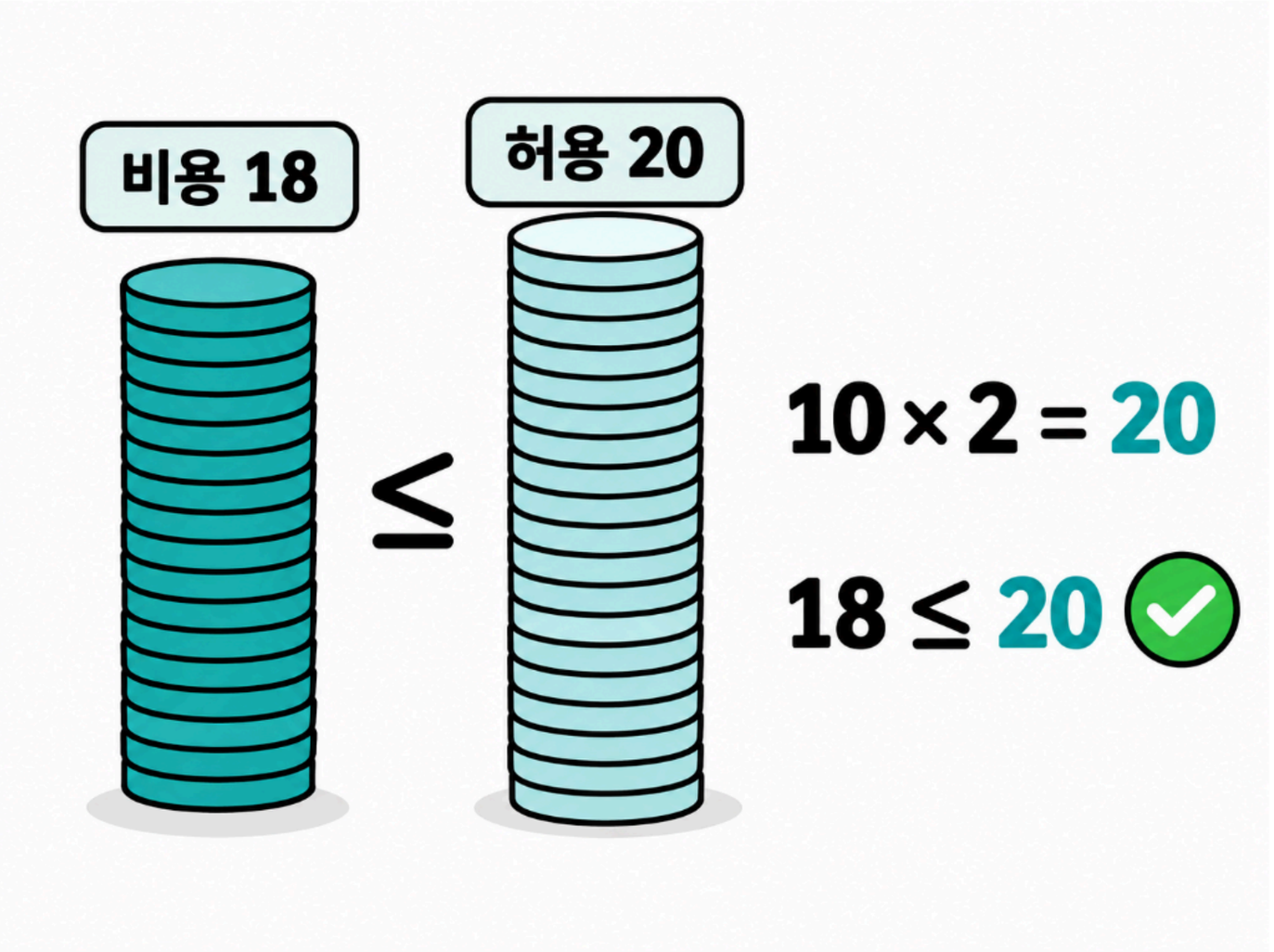
작은 예

가장 싼 비용이 10이고 두 배까지 허용합니다.
허용되는 최대 비용은 10×2=20입니다.

반환할 값

비용이 18이면 허용 범위 안이므로 1, 21이면 0을 반환합니다.

10-1 그림으로 보는 예



최적 비용 10
두 배까지 허용
18은 20 이하
따라서 1 반환

10-1 하나씩 계산한 결과

구한 비용 cost	허용 비용 opt×factor	비교	답
18	20	18≤20	1
20	20	20≤20	1
21	20	21>20	0

정확히 두 배인 20도 허용합니다.

10-1 계산 순서

허용 비용 계산

최적 비용 opt에 허용 배수 factor를 곱합니다.

비교

구한 비용 cost가 이 값 이하인지 봅니다.

반환

이하면 1, 초과하면 0입니다.

10-1 변수와 코드의 뜻

변수	예제의 값
cost	구한 해의 비용 18
opt	가능한 가장 싼 비용 10
factor	허용 배수 2
cost <= opt*factor	18≤20이므로 참

근사 알고리즘에서는 빨리 구한 답의 비용이 최적 비용보다 얼마나 큰지도 봅니다.

10-1 함수에 들어오는 값

함수

within_factor(cost, opt, factor)

입력 조건

1≤opt≤cost≤1,000,000, 허용 배수 factor는 정수 1~100입니다.

10-1 Python

```
def within_factor(cost, opt, factor):
    return int(cost <= factor*opt)
```

시간 O(1), 추가 공간 O(1)

10-1 C

```
int within_factor(int cost, int opt, int factor) {
    return cost<=factor*opt;
}
```

시간 O(1), 추가 공간 O(1)

10-1 다른 입력도 확인합니다

cost, opt, factor	답	이유
18,10,2	1	20 이하
20,10,2	1	같아도 허용
21,10,2	0	20 초과

근사 알고리즘에서는 빨리 구한 답의 비용이 최적 비용보다 얼마나 큰지도 봅니다.

10-2 가장 한가한 기계에 작업 배정

구할 것

들어오는 작업을 그때 가장 한가한 기계에 맡깁니다.

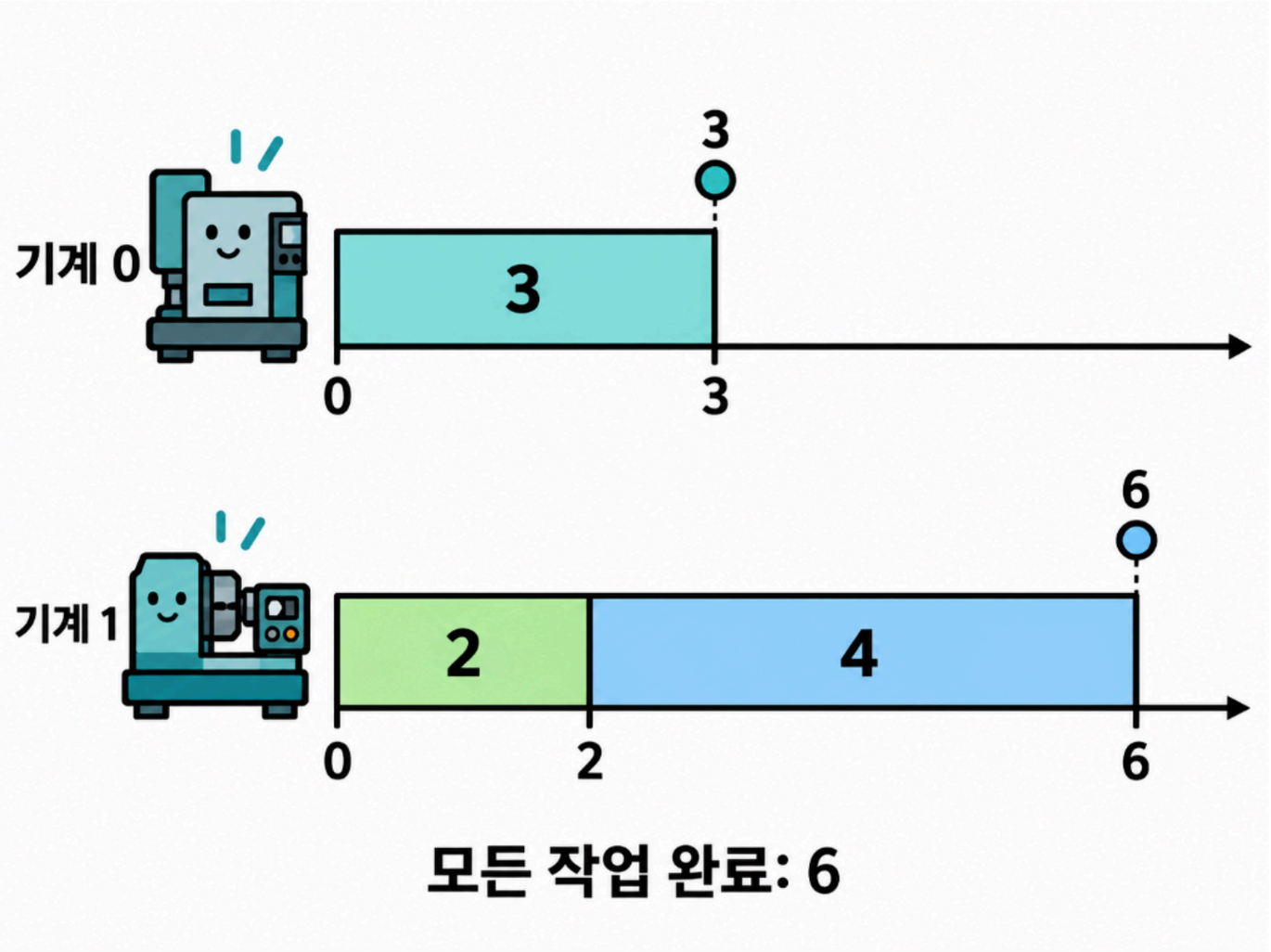
작은 예

기계 두 대에 길이 3,2,4인 작업을 이 순서대로 맡깁니다.
처음에는 두 기계 모두 누적 작업 시간이 0입니다.

반환할 값

모든 작업이 끝나는 시각 6을 반환합니다.

10-2 그림으로 보는 예



작업 3은 기계0
작업 2는 기계1
작업 4도 기계1
기계별 합은 3과 6

10-2 하나씩 계산한 결과

이번 작업 길이	작업 전 기계별 합	맡길 기계	작업 후 합
3	[0,0]	0	[3,0]
2	[3,0]	1	[3,2]
4	[3,2]	1	[3,6]

마지막 작업 직전에는 기계1의 합 2가 기계0의 합 3보다 작습니다.

10-2 계산 순서

작업 순서는 유지

jobs를 앞에서부터 하나씩 읽습니다.

가장 작은 누적 시간

load가 가장 작은 기계에 작업 시간을 더합니다.

완료 시각

기계별 누적 시간 중 가장 큰 값입니다.

10-2 변수와 코드의 뜻

변수	뜻
load[i]	기계 i에 지금까지 맡긴 시간의 합
machine	이번 작업을 맡길 기계 번호
load[machine] += time	선택한 기계의 합에 이번 작업 시간 추가
max(load)	가장 늦게 끝나는 기계의 완료 시각

누적 시간이 같으면 번호가 작은 기계를 먼저 고릅니다.

10-2 함수에 들어오는 값

함수

list_scheduling(jobs, m)

입력 조건

기계 1~10대, 작업 0~1000개, 각 작업 시간 0~1,000,000입니다.

10-2 Python

```
def list_scheduling(jobs, m):
    load = [0]*m
    for time in jobs:
        machine = 0
        for i in range(1, m):
            if load[i] < load[machine]:
                machine = i
        load[machine] += time
    return max(load)
```

작업 n개, 기계 m대일 때 시간 O(nm), 공간 O(m)

10-2 C

```
int list_scheduling(const int jobs[], int n, int m) {
    int load[10]={0};
    for (int i=0; i<n; ++i) {
        int machine=0;
        for (int j=1; j<m; ++j)
            if (load[j]<load[machine]) machine=j;
        load[machine]+=jobs[i];
    }
    int answer=0;
    for (int j=0; j<m; ++j)
        if (load[j]>answer) answer=load[j];
    return answer;
}
```

작업 n개, 기계 m대일 때 시간 O(nm), 공간 O(m)

10-2 다른 입력도 확인합니다

상황	답	확인할 점
작업 없음	0	바로 완료
기계1대, 작업 [3,2,4]	9	한 기계가 전부 처리
기계2대, 작업 [3,2,4]	6	주어진 순서 유지

누적 시간이 같으면 번호가 작은 기계를 먼저 고릅니다.

10-3 간선의 양 끝을 고르는 정점 커버

구할 것

아직 고르지 않은 두 점을 잇는 선을 만나면 두 점을 모두 고릅니다.

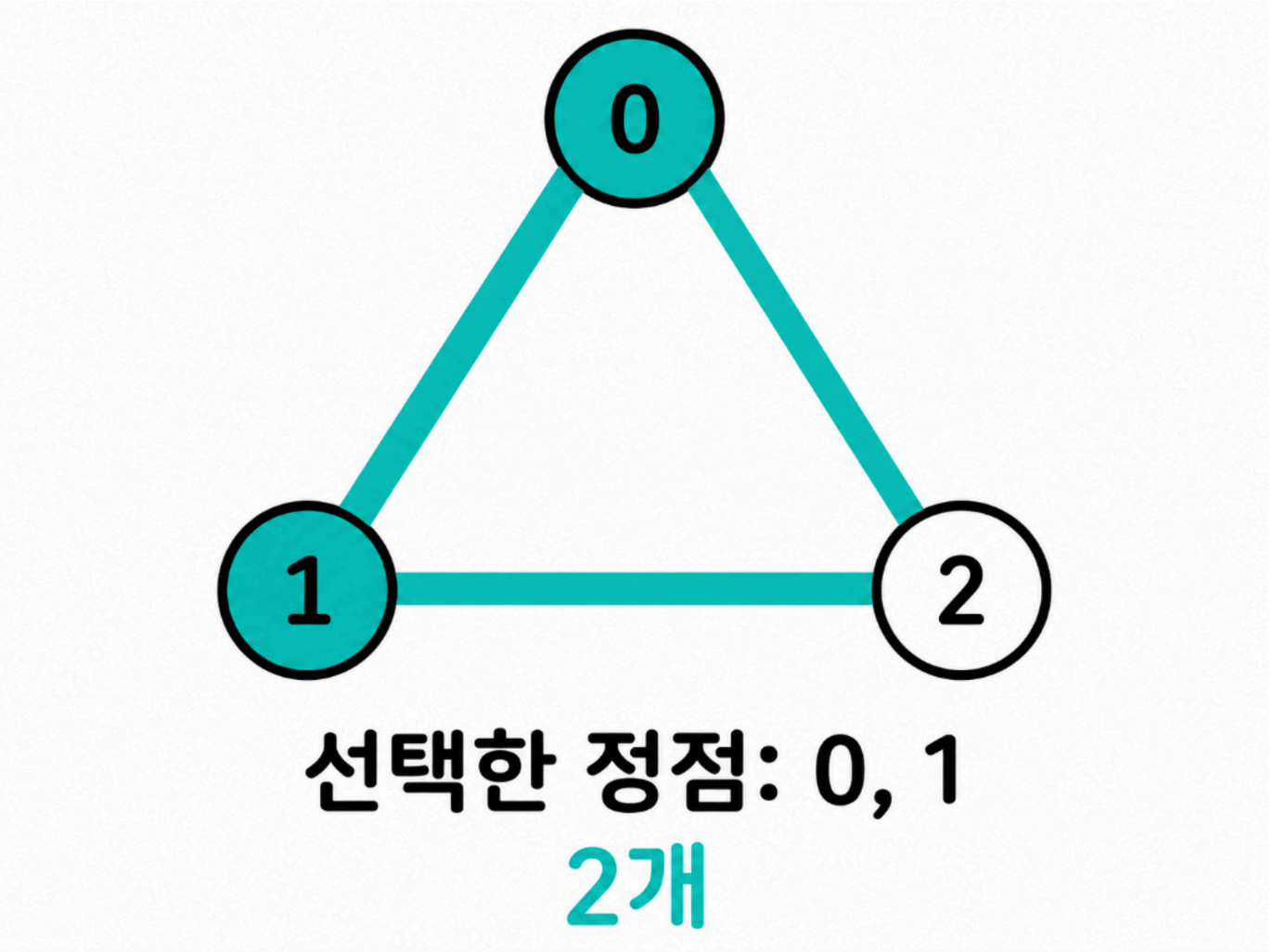
작은 예

삼각형의 선을 [0–1, 1–2, 0–2] 순서로 봅니다.
첫 선에서 0과 1을 고르면 나머지 두 선도 선택한 점과 닿습니다.

반환할 값

마지막에 고른 점의 개수 2를 반환합니다.

10-3 그림으로 보는 예



선 0–1을 먼저 봄
점0과 점1을 고름
다른 두 선도 덮임
고른 점은 두 개

10-3 하나씩 계산한 결과

이번 선	현재 고른 점	처리	처리 후
0–1	없음	두 끝점을 모두 고름	0,1
1–2	0,1	1을 이미 골랐으므로 넘어감	0,1
0–2	0,1	0을 이미 골랐으므로 넘어감	0,1

9-2의 검사와 달리, 이번에는 규칙에 따라 점을 직접 고릅니다.

왜 두 배 이내인가요?

두 끝점을 새로 고른 선끼리는 서로 점이 겹치지 않습니다.

이런 선이 두 개라면

어떤 정점 커버도 각 선에 하나씩, 적어도 두 점을 골라야 합니다.

이 절차의 선택

각 선의 두 끝을 골라 총 네 점을 고릅니다.

같은 계산을 일반화

선이 k개이면 최소 k개가 필요하고, 우리는 2k개를 고릅니다.

10-3 계산 순서

간선 입력 순서

edges의 선을 앞에서부터 봅니다.

양 끝 모두 미선택

두 점을 모두 선택하고 개수에 2를 더합니다.

하나라도 이미 선택

그 선은 이미 덮었으므로 넘어갑니다.

10-3 변수와 코드의 뜻

코드	뜻
u, v	이번 선의 양 끝 점 번호
chosen[u]	점 u를 골랐으면 True
not chosen[u] and not chosen[v]	두 점 모두 아직 고르지 않았음
answer += 2	새 점 두 개를 골랐으므로 개수에 2를 더함

이 절차가 만든 점의 수는 최소 정점 커버 크기의 두 배 이내입니다.

10-3 함수에 들어오는 값

함수

cover_size(n, edges)

입력 조건

점 1~100개, 선 0~1000개. 점 번호는 0~n-1이며 자기 자신을 잇는 선은 없습니다.

10-3 Python

```
def cover_size(n, edges):
    chosen = [False]*n
    answer = 0
    for u, v in edges:
        if not chosen[u] and not chosen[v]:
            chosen[u] = True
            chosen[v] = True
            answer += 2
    return answer
```

간선 m개일 때 시간 O(n+m), 공간 O(n)

10-3 C

```
int cover_size(int n, const int edges[][2], int m) {
    int chosen[100]={0}, answer=0;
    for (int i=0; i<m; ++i) {
        int u=edges[i][0], v=edges[i][1];
        if (!chosen[u] && !chosen[v]) {
            chosen[u]=chosen[v]=1;
            answer+=2;
        }
    }
    return answer;
}
```

간선 m개일 때 시간 O(n+m), 공간 O(n)

10-3 다른 입력도 확인합니다

상황	답	이유
선 없음	0	고를 점 없음
선 하나 0–1	2	두 끝 모두 선택
서로 떨어진 선 두 개	4	각 선에서 두 점씩 선택

이 절차가 만든 점의 수는 최소 정점 커버 크기의 두 배 이내입니다.

10-4 새 원소를 가장 많이 덮는 집합 선택

구할 것

아직 덮지 못한 원소를 가장 많이 덮는 집합을 반복해서 고릅니다.

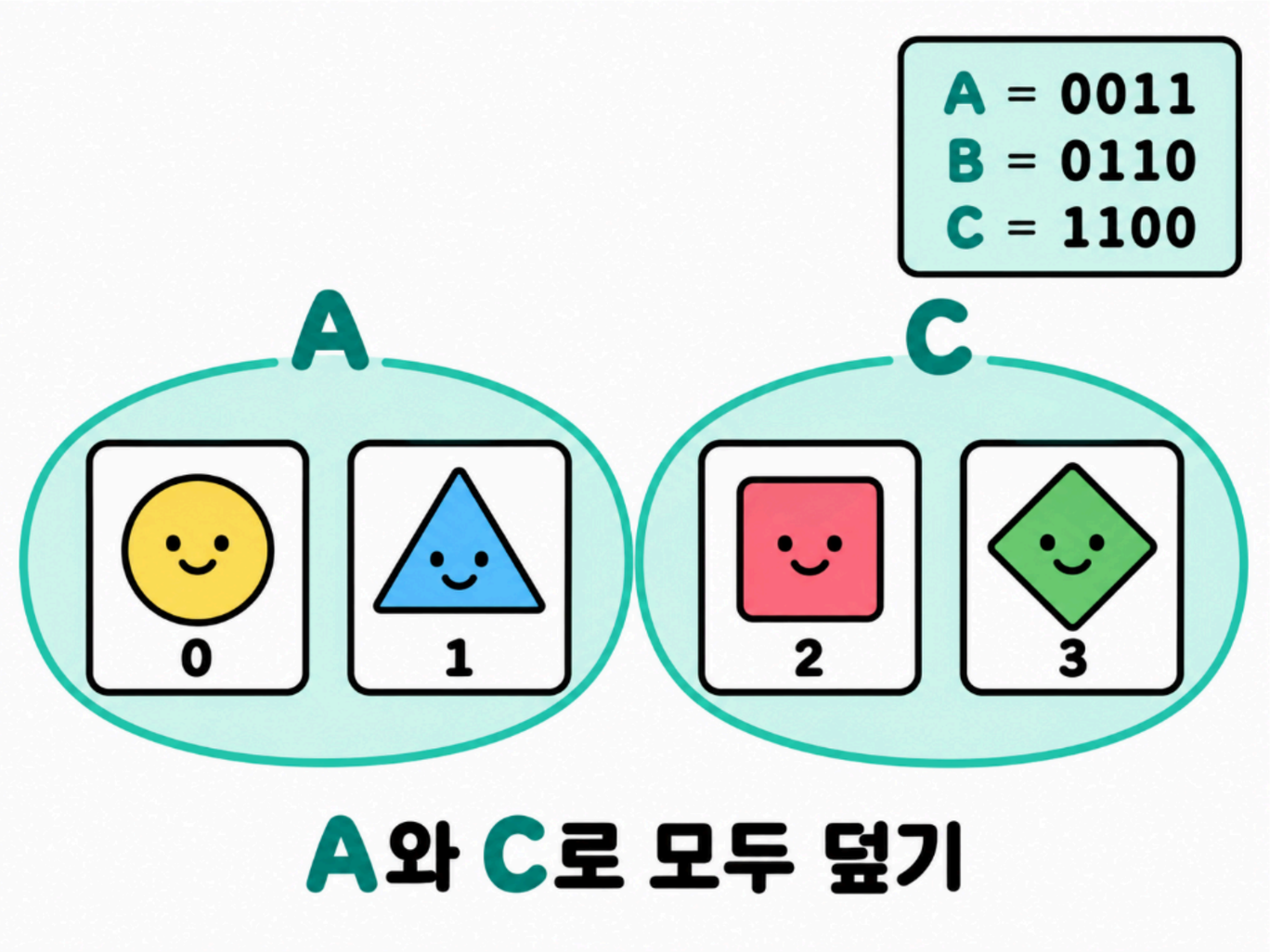
작은 예

전체 원소는 0,1,2,3입니다.
A={0,1}, B={1,2}, C={2,3} 중에서 고릅니다.

반환할 값

A를 고른 뒤 C를 고르면 전부 덮습니다. 선택한 집합 수 2를 반환합니다.

10-4 그림으로 보는 예



처음에는 A 선택
남은 원소는 2,3
이 둘을 덮는 C 선택
두 번으로 완료

10-4 하나씩 계산한 결과

남은 원소	A가 새로 덮음	B가 새로 덮음	C가 새로 덮음	선택
0,1,2,3	2	2	2	먼저 나온 A
2,3	0	1	2	C
없음	0	0	0	2회로 완료

동률이면 입력에서 앞에 있는 집합을 고릅니다.

집합을 정수 3, 6, 12로 적는 이유

오른쪽부터 원소 0,1,2,3의 포함 여부를 적습니다.

집합	원소3	원소2	원소1	원소0	정수
A={0,1}	0	0	1	1	3
B={1,2}	0	1	1	0	6
C={2,3}	1	1	0	0	12
전체	1	1	1	1	15

가중치는 오른쪽부터 1,2,4,8입니다. A는 1+2=3입니다.

한 번 고른 뒤 남은 원소의 변화

전체 1111에서 A의 0011을 제거합니다.

계산	비트 표시	뜻
처음 remaining	1111	0,1,2,3이 남음
선택한 A	0011	0,1을 덮음
제거 후 remaining	1100	2,3만 남음
남은 것과 B의 공통 부분	1100 & 0110 = 0100	B는 2 하나만 새로 덮음

&는 두 표시가 모두 1인 자리만 남기는 연산입니다.

10-4 계산 순서

새로 덮는 개수

각 집합에서 아직 남은 원소만 셉니다.

가장 큰 집합 선택

새로 덮는 수가 가장 큰 집합을 고르고 그 원소들을 지웁니다.

끝나는 조건

전부 덮으면 횟수 반환. 아무것도 더 덮지 못하면 -1입니다.

10-4 변수와 코드의 뜻

변수	뜻
remaining	아직 덮지 못한 원소들의 표시
remaining & mask	현재 집합이 새로 덮을 수 있는 원소들
bit_count()	1로 표시된 원소의 수
remaining &= ~best	고른 집합의 원소들을 남은 목록에서 제거

비트마스크는 원소가 들어 있는지 0과 1로 적은 정수입니다.

10-4 함수에 들어오는 값

함수

greedy_set_cover(n, sets)

입력 조건

원소 1~20개, 집합 0~30개. 각 집합은 정수 0~2^n–1로 표시합니다.

10-4 Python

```
def greedy_set_cover(n, sets):
    remaining = (1 << n)-1
    count = 0
    while remaining:
        best, gain = 0, 0
        for mask in sets:
            added = (mask & remaining).bit_count()
            if added > gain:
                best, gain = mask, added
        if gain == 0: return -1
        remaining &= ~best
        count += 1
    return count
```

단순 비트 개수 계산 기준 시간 $O(mn^2)$, 공간 $O(1)$

10-4 C (1/2)

```
static int count_bits(int x) {
    int n=0;
    while (x) {
        n+=x&1;
        x>>=1;
    }
    return n;
}
int greedy_set_cover(int n, const int sets[], int m) {
    int remaining=(1<<n)-1, count=0;
    while (remaining) {
        int best=0, gain=0;
        for (int i=0; i<m; ++i) {
```

다음 쪽에서 이어집니다.

10-4 C (2/2)

```
            int added=count_bits(sets[i]&remaining);
            if (added>gain) {
                best=sets[i];
                gain=added;
            }
        }
        if (!gain) return -1;
        remaining&=~best;
            ++count;
    }
    return count;
}
```

단순 비트 개수 계산 기준 시간 $O(mn^2)$, 공간 $O(1)$

10-4 다른 입력도 확인합니다

상황	답	이유
전체를 담은 집합 하나	1	한 번에 전부 덮음
어느 집합에도 없는 원소 존재	-1	전부 덮는 것이 불가능
위 A,B,C 예제	2	A 다음 C

비트마스크는 원소가 들어 있는지 0과 1로 적은 정수입니다.

10장의 계산을 한 문장으로

문제	구하는 값
10-1	구한 비용이 허용한 배수 안에 들어오는지 확인합니다.
10-2	들어오는 작업을 그때 가장 한가한 기계에 맡깁니다.
10-3	아직 고르지 않은 두 점을 잇는 선을 만나면 두 점을 모두 고릅니다.
10-4	아직 덮지 못한 원소를 가장 많이 덮는 집합을 반복해서 고릅니다.

최적 답을 구하기 어려울 때, 빨리 답을 구하면서 얼마나 나쁠 수 있는지도 봅니다.